

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

B ANIL KUMAR (1BM25CS421)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **BANIL KUMAR (1BM25CS421)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr.Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	1.Binary Search (Leetcode -704) 2.Sort List(Leetcode – 148)	05-09
2	Missing Number (Leetcode – 268)	10-11
3	1.Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. 2.Sort Colors (Leetcode – 75)	12-17
4	1.Sort a given set of N integer elements using Quick Sort technique and compute its time taken. 2.Sort Array(Leetcode – 912)	18-23
5	1.Implement Fractional Knapsack using Greedy technique.(Perform Optimal solution 2.Buy Two Chocolates (Leetcode -2706)	24-28
6	1.Implement All Pair Shortest paths problem using Floyd’s algorithm. 2.From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra’s algorithm. 3.Can Place Flowers(Leetcode – 605)	29-35
7	1. Implement the 0/1 Knapsack problem using dynamic programming. 2.Target Sum(Leetcode – 494)	36-40
8	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	41-43
9	1.Write program to obtain the Topological ordering of vertices in a given digraph. 2.Course Schedule (Leetcode – 207)	44-49
10	1. Find the minimum cost spanning tree of a given undirected graph using Prim’s algorithm. 2. Find the minimum cost spanning tree of a given undirected graph using Kruskal’s algorithm.	50-55
11	i)Implement Johnson Trotter algorithm to generate permutations. ii) Implement “N-Queens Problem” using Backtracking.	56-60

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Week – 1

1.Binary Search (Leetcode -704)

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [-1,0,3,5,9,12], target = 9

Output: 4

Explanation: 9 exists in nums and its index is 4

Example 2:

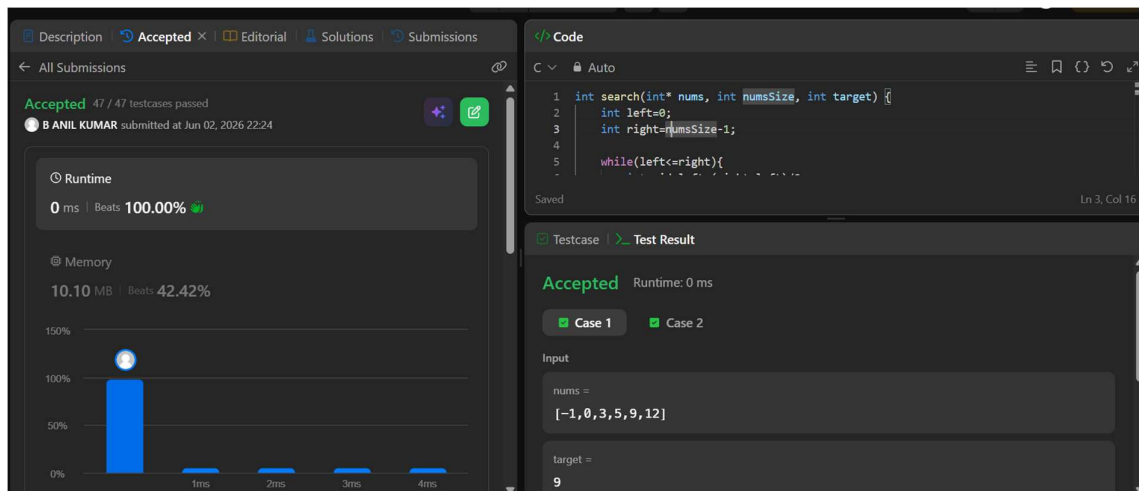
Input: nums = [-1,0,3,5,9,12], target = 2

Output: -1

Explanation: 2 does not exist in nums so return -1

C Code :

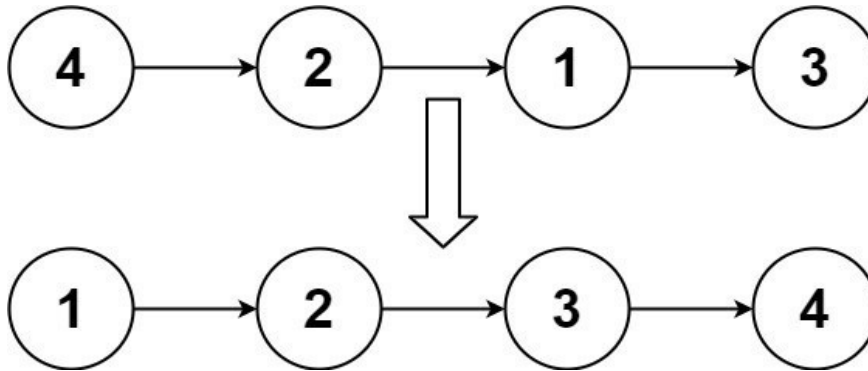
```
int search(int* nums, int numsSize, int target) {  
  
    int left=0;  
  
    int right=numsSize-1;  
  
    while(left<=right){  
  
        int mid=left+(right-left)/2;  
  
        if(nums[mid]==target){  
  
            return mid;  
  
        } else if(nums[mid]<target){  
  
            left=mid+1;  
  
        } else {  
  
            right=mid-1;  
  
        }  
  
    }  
  
    return -1;  
}
```



2.Sort List(Leetcode – 148)

Given the head of a linked list, return *the list after sorting it in ascending order*.

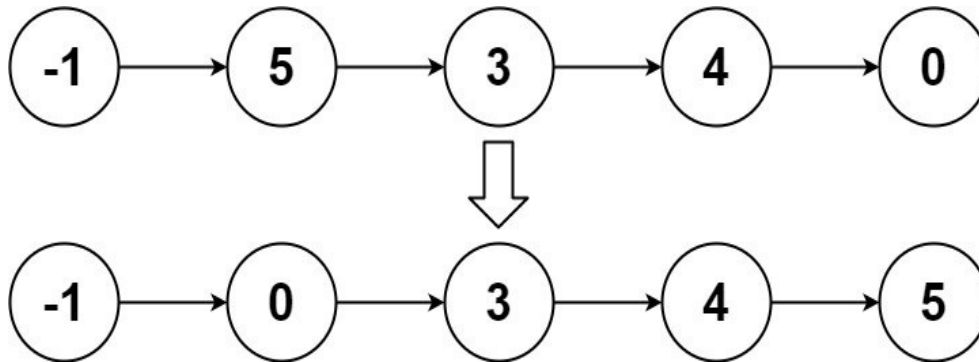
Example 1:



Input: head = [4,2,1,3]

Output: [1,2,3,4]

Example 2 :



Input: head = [-1,5,3,4,0]

Output: [-1,0,3,4,5]

C Code:-

```
struct ListNode* merge(struct ListNode* l1, struct ListNode* l2) {
    struct ListNode dummy;
    struct ListNode* temp = &dummy;
    dummy.next = NULL;
    while (l1 != NULL && l2 != NULL) {
        if (l1->val <= l2->val) {
            temp->next = l1;
            l1 = l1->next;
        } else {
            temp->next = l2;
            l2 = l2->next;
        }
        temp = temp->next;
    }
    if (l1 != NULL)
        temp->next = l1;
    if (l2 != NULL)
        temp->next = l2;
    return dummy.next;
}

struct ListNode* sortList(struct ListNode* head) {
    if (head == NULL || head->next == NULL) {
        return head;
    }
    struct ListNode* slow = head;
    struct ListNode* fast = head;
    struct ListNode* prev = NULL;
    while (fast != NULL && fast->next != NULL) {
        prev = slow;
        slow = slow->next;
```



```

    fast = fast->next->next;
}

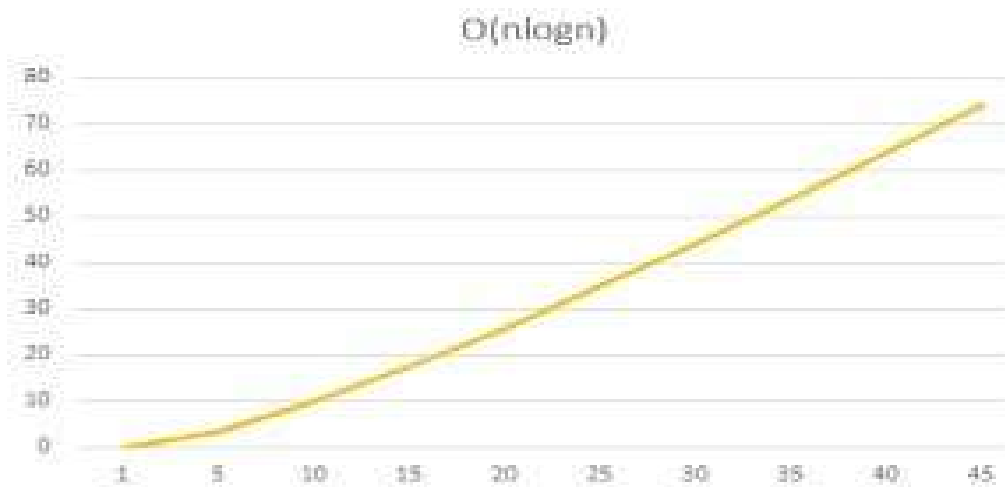
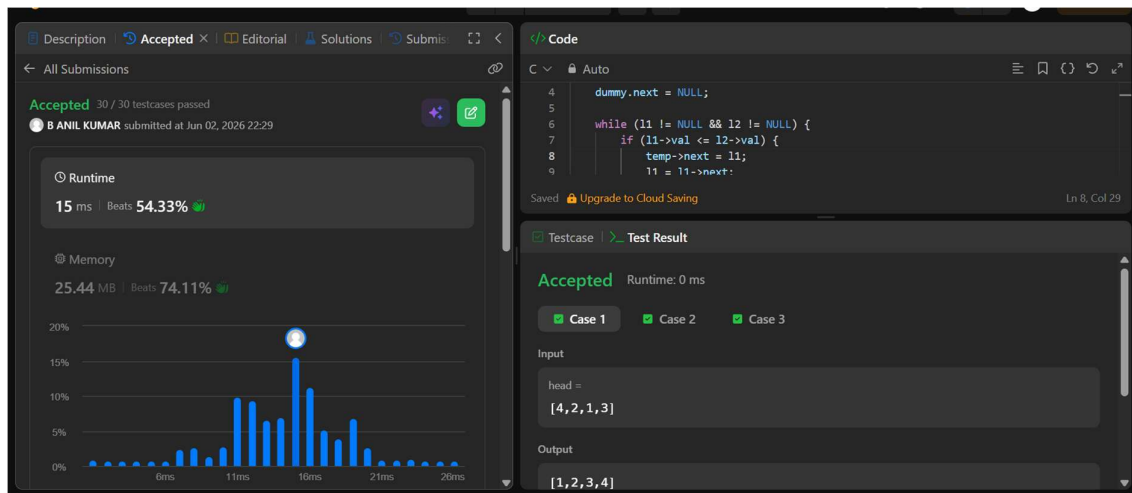
prev->next = NULL;

struct ListNode* left = sortList(head);

struct ListNode* right = sortList(slow);

return merge(left, right);
}

```



Week – 2

1. Missing Number (Leetcode – 268)

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return *the only number in the range that is missing from the array*.

Example 1:

Input:

`nums=[3,0,1]`

Output: 2

Explanation:

$n = 3$ since there are 3 numbers, so all numbers are in the range $[0,3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input:

`nums = [0,1]`

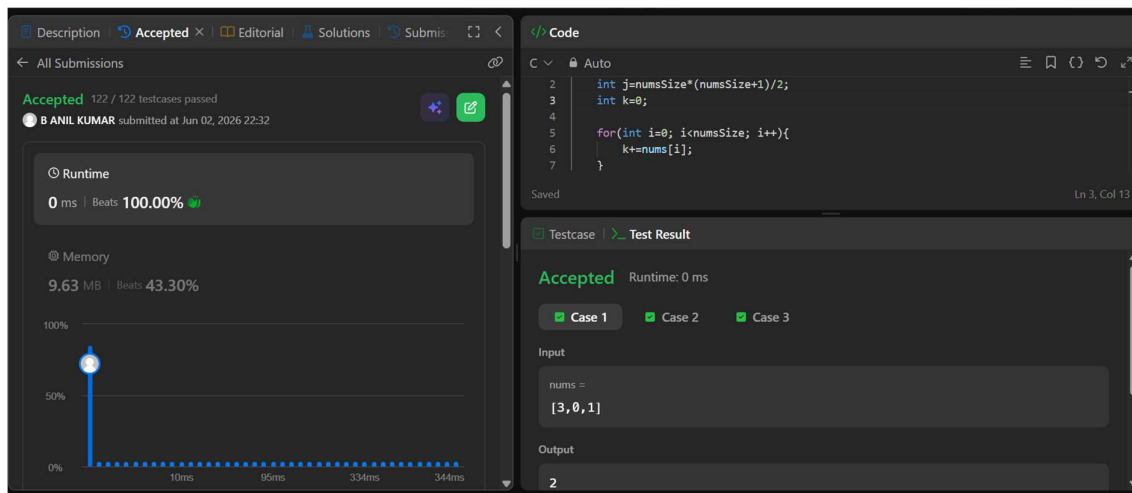
Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

C Code:-

```
int missingNumber(int* nums, int numsSize) {  
    int j=numsSize*(numsSize+1)/2;  
    int k=0;  
    for(int i=0; i<numsSize; i++){  
        k+=nums[i];  
    }  
    return j-k;  
}
```



Week – 3

1.Merge Sort

Sort a given set of N integer elements using Merge Sort technique and compute its time taken.
Run the program for different values of N and record the time taken to sort. (5 Marks)

C Code:-

```
#include<stdio.h>

void Merg(int arr[], int x, int m, int r){

    int n1=m-x+1, n2=r-m;

    int L[n1], R[n2];

    for(int i=0; i<n1; i++){

        L[i]=arr[x+i];

    }

    for(int j=0; j<n2; j++){

        R[j]=arr[m+1+j];

    }

    int i=0, j=0, k=x;

    while(i<n1 && j<n2){

        if(L[i]<=R[j]){

            arr[k++]=L[i++];

        }else{

            arr[k++]=R[j++];

        }

    }

    while(i<n1){
```

```

        arr[k++]=L[i++];
    }
    while(j<n2){
        arr[k++]=R[j++];
    }
}

void MergSort(int arr[], int x, int r){
    if(x < r){
        int m=(x+r)/2;
        MergSort(arr, x, m);
        MergSort(arr, m+1, r);
        Merg(arr, x, m, r);
    }
}

int main(){
    int n;
    printf("Enter the Number of Elements : ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the Array of Elements : ");
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    MergSort(arr, 0, n-1);
    printf("Sorted List : ");
    for(int i=0; i<n; i++){

```

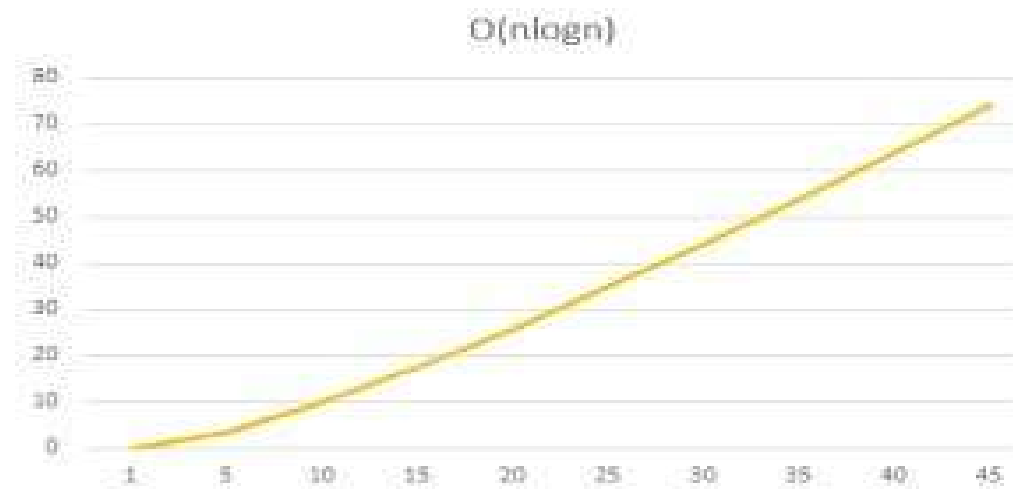
```
    printf("%d ", arr[i]);  
}  
}
```

Output:-

Enter the Number of Elements : 6

Enter the Array of Elements : 9 4 8 2 1 5

Sorted List : 1 2 4 5 8 9



2.Sort Colors (Leetcode – 75)

Given an array `nums` with `n` objects colored red, white, or blue, sort them [in-place](#) so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`

Output: `[0,1,2]`

```

void sortColors(int* nums, int numsSize) {

    int n = numsSize;

    for (int i = 0; i < n - 1; i++) {

        for (int j = 0; j < n - i - 1; j++) {

            if (nums[j] > nums[j + 1]) {

                int temp = nums[j];

                nums[j] = nums[j + 1];

                nums[j + 1] = temp;

            }

        }

    }

}

```

Problem List < > Submit

Description | Accepted x | Editorial | Solutions | Submissic

All Submissions

Accepted 89 / 89 testcases passed

ANIL KUMAR submitted at Jun 02, 2026 22:39

Runtime

0 ms | Beats 100.00%

Memory

10.63 MB | Beats 4.18%

Code

```

1 void sortColors(int* nums, int numsSize) {
2     int n = numsSize;
3     for (int i = 0; i < n - 1; i++) {
4         for (int j = 0; j < n - i - 1; j++) {
5             if (nums[j] > nums[j + 1]) {

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

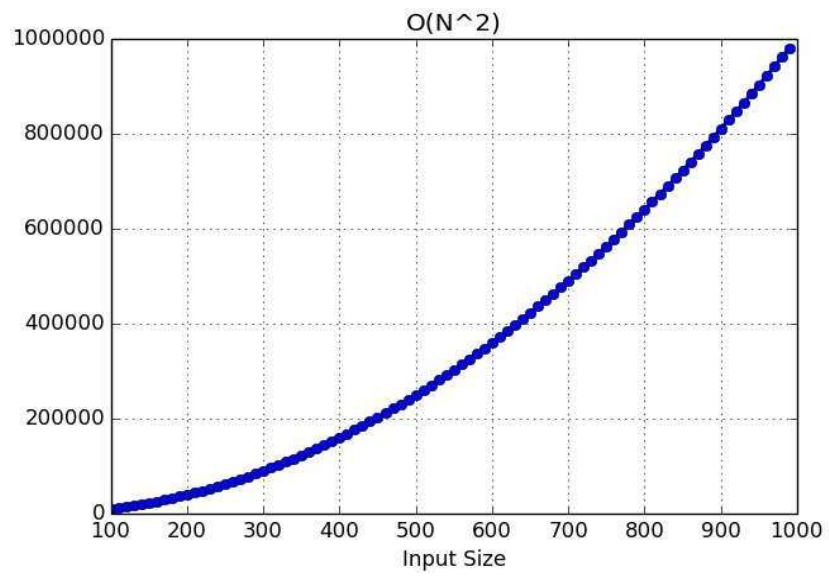
Input

nums =

[2,0,2,1,1,0]

Output

[0,0,1,1,2,2]



Week – 4

1.Quick Sort

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.
(5 Marks)

C Code:-

```
#include<stdio.h>

#include<time.h>

void swap(int *a, int *b){

    int temp=*a;

    *a=*b;

    *b=temp;

}

int partition(int arr[], int low, int high){

    int pivot=arr[high];

    int i=low-1;

    for(int j=low; j<high; j++){

        if(arr[j]<=pivot){

            i++;

            swap(&arr[i], &arr[j]);

        }

    }

    swap(&arr[i+1], &arr[high]);

    return i+1;

}

void QuickSort(int arr[], int low, int high){

    if(low<high){
```

```

        int pi=partition(arr, low, high);

        QuickSort(arr, low, pi-1);

        QuickSort(arr, pi+1, high);

    }
}

int main(){

    int n;

    printf("Enter the Number of Elements : ");

    scanf("%d", &n);

    int arr[n];

    printf("Enter the Array of Elements : ");

    for(int i=0; i<n; i++){

        scanf("%d", &arr[i]);

    }

    clock_t start, end;

    start=clock();

    QuickSort(arr, 0, n-1);

    end=clock();

    printf("Sorted List : ");

    for(int i=0; i<n; i++){

        printf("%d ", arr[i]);

    }

    double time_taken=((double)(end-start))/CLOCKS_PER_SEC;

    printf("Time Taken : %f", time_taken);

}

```

Output:-

Enter the Number of Elements : 10

Enter the Array of Elements : 3

4 5 2 1 7 8 6 0 9

Sorted List : 0 1 2 3 4 5 6 7 8 9



2.Sort Array(Leetcode – 912)

Given an array of integers nums, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: nums = [5,2,3,1]

Output: [1,2,3,5]

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: nums = [5,1,1,2,0,0]

Output: [0,0,1,1,2,5]

Explanation: Note that the values of nums are not necessarily unique.

C Code:-

```
void merge(int nums[], int lb, int mid, int ub) {
    int i = lb;
    int j = mid + 1;
    int k = 0;
    int temp[50000];
    while (i <= mid && j <= ub) {
        if (nums[i] <= nums[j]) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    while (i <= mid) {
        temp[k++] = nums[i++];
    }
    while (j <= ub) {
        temp[k++] = nums[j++];
    }
    for (i = lb, k = 0; i <= ub; i++, k++) {
        nums[i] = temp[k];
    }
}

void mergesort(int nums[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergesort(nums, l, mid);
        mergesort(nums, mid + 1, r);
        merge(nums, l, mid, r);
    }
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
```

```

*returnSize = numsSize;

mergesort(nums, 0, numsSize - 1);

return nums;
}

```

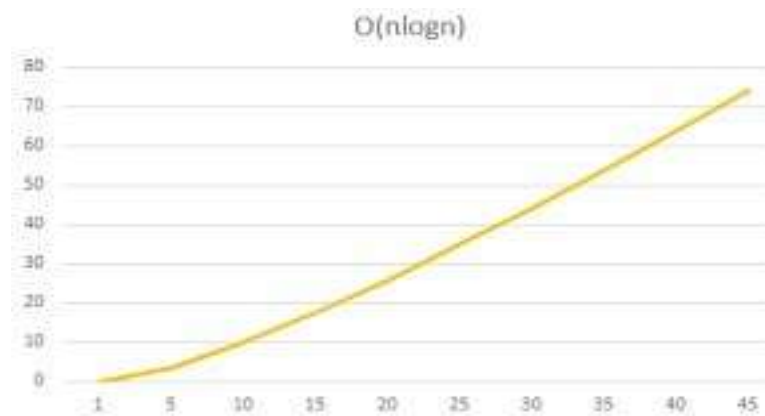
The screenshot shows a code editor with a C++ implementation of merge sort. The code is as follows:

```

1 void merge(int nums[], int lb, int mid, int ub) {
2     int i = lb;
3     int j = mid + 1;
4     int k = 0;
5     int temp[50000];
6 }

```

The test results show that the code is "Accepted" with a runtime of 194 ms and memory usage of 59.14 MB. The input is `nums = [5, 2, 3, 1]` and the output is `[1, 2, 3, 5]`.



Week – 5

1.Knapsack Problem

Implement Fractional Knapsack using Greedy technique.(Perform Optimal solution)(5 Marks)

C Code:-

```
#include <stdio.h>

struct Item
{
    int weight;
    int profit;
    float ratio;
};

void sort(struct Item arr[], int n)
{
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = 0; j < n - i - 1; j++)
        {
            if(arr[j].ratio < arr[j + 1].ratio)
            {
                struct Item temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```



```

}

int main()

{
    int n, capacity;

    printf("Enter the number of items: ");

    scanf("%d", &n);

    struct Item arr[n];

    printf("Enter profit and weight of each item:\n");

    for(int i = 0; i < n; i++)

    {
        scanf("%d %d", &arr[i].profit, &arr[i].weight);

        arr[i].ratio = (float)arr[i].profit / arr[i].weight;

    }

    printf("Enter knapsack capacity: ");

    scanf("%d", &capacity);

    sort(arr, n);

    float totalProfit = 0.0;

    for(int i = 0; i < n; i++)

    {
        if(capacity >= arr[i].weight)

        {
            capacity -= arr[i].weight;

            totalProfit += arr[i].profit;

        }

        else

        {

```

```
        totalProfit += arr[i].ratio * capacity;

        break;
    }
}

printf("Maximum Profit = %.2f\n", totalProfit);

return 0;
}
```

Output:-

Enter the number of items: 3

Enter profit and weight of each item:

60 10

100 20

120 30

Enter knapsack capacity: 50

Maximum Profit = 240.00

2.Buy Two Chocolates (Leetcode -2706)

You are given an integer array `prices` representing the prices of various chocolates in a store. You are also given a single integer `money`, which represents your initial amount of money.

You must buy **exactly** two chocolates in such a way that you still have some **non- negative** leftover money. You would like to minimize the sum of the prices of the two chocolates you buy.

Return *the amount of money you will have leftover after buying the two chocolates*. If there is no way for you to buy two chocolates without ending up in debt, return `money`. Note that the leftover must be non-negative.

Example 1:

Input: `prices = [1,2,2]`, `money = 3`

Output: 0

Explanation: Purchase the chocolates priced at 1 and 2 units respectively. You will have $3 - 3 = 0$ units of money afterwards. Thus, we return 0.

Example 2:

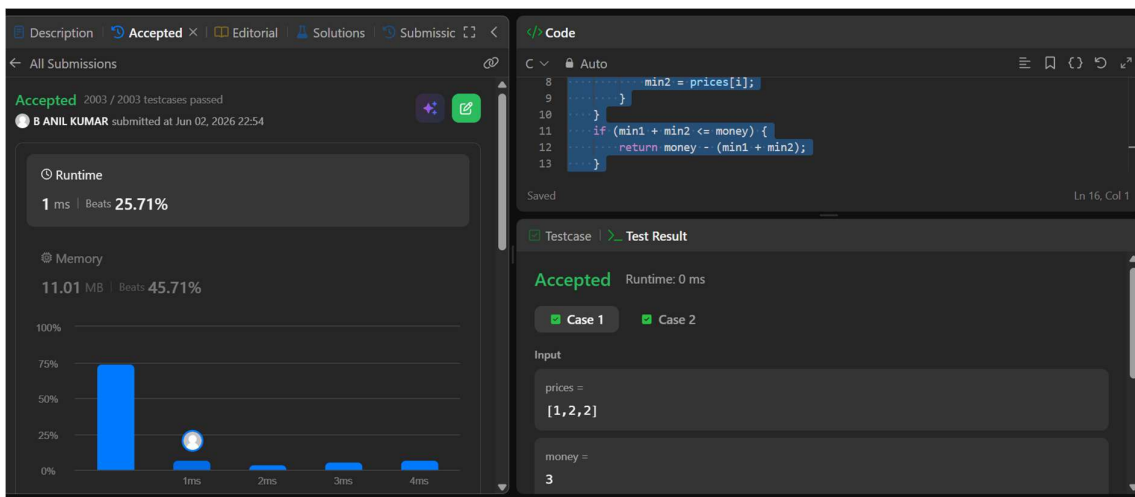
Input: `prices = [3,2,3]`, `money = 3`

Output: 3

Explanation: You cannot buy 2 chocolates without going in debt, so we return 3.

C Code:-

```
int buyChoco(int* prices, int pricesSize, int money) {  
    int min1 = INT_MAX, min2 = INT_MAX;  
    for (int i = 0; i < pricesSize; i++) {  
        if (prices[i] < min1) {  
            min2 = min1;  
            min1 = prices[i];  
        } else if (prices[i] < min2) {  
            min2 = prices[i];  
        }  
    }  
    if (min1 + min2 <= money) {  
        return money - (min1 + min2);  
    }  
    return money;  
}
```



Week – 6

1.Floyd's Algorithm

Implement All Pair Shortest paths problem using Floyd's algorithm.

C Code:-

```
#include <stdio.h>

#define INF 99999

int main()
{
    int n;

    scanf("%d", &n);

    int graph[n][n];

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] == -1)
                graph[i][j] = INF;

            if (i == j)
                graph[i][j] = 0;
        }
    }

    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (graph[i][k] != INF &&
                    graph[k][j] != INF &&
                    graph[i][k] + graph[k][j] < graph[i][j])
                {
```

```

        graph[i][j] = graph[i][k] + graph[k][j];
    }
}
}
}
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (graph[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", graph[i][j]);
    }
    printf("\n");
}
return 0;
}

```

Output:-

```

4
0 5 -1 10
-1 0 3 -1
-1 -1 0 1
-1 -1 -1 0

0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

```

2. Dijkstra's Algorithm

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

C Code:-

```
#include <stdio.h>

#define INF 99999

int main()
{
    int n;

    scanf("%d", &n);

    int graph[n][n];

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    int src;

    scanf("%d", &src);

    int dist[n], visited[n];

    for(int i = 0; i < n; i++)
    {
        dist[i] = INF;

        visited[i] = 0;
    }
```

```

dist[src] = 0;

for(int count = 0; count < n - 1; count++)
{
    int min = INF, u = -1;

    for(int i = 0; i < n; i++)
    {
        if(!visited[i] && dist[i] < min)
        {
            min = dist[i];
            u = i;
        }
    }

    if(u == -1)
        break;

    visited[u] = 1;

    for(int v = 0; v < n; v++)
    {
        if(!visited[v] &&
            graph[u][v] != 0 &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v])
        {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

```



```
    for(int i = 0; i < n; i++)  
        printf("%d ", dist[i]);  
    return 0;  
}
```

Output:-

```
5  
0 10 0 30 100  
10 0 50 0 0  
0 50 0 20 10  
30 0 20 0 60  
100 0 10 60 0  
0  
  
0 10 50 30 60
```

3.Can Place Flowers(Leetcode – 605)

You have a long flowerbed in which some of the plots are planted, and some are not. However, flowers cannot be planted in **adjacent** plots.

Given an integer array flowerbed containing 0's and 1's, where 0 means empty and 1 means not empty, and an integer n, return true *if n new flowers can be planted in the flowerbed without violating the no-adjacent-flowers rule and false otherwise.*

Example 1:

Input: flowerbed = [1,0,0,0,1], n = 1

Output: true

Example 2:

Input: flowerbed = [1,0,0,0,1], n = 2

Output: false

C Code:-

```
bool canPlaceFlowers(int* flowerbed, int flowerbedSize, int n) {  
  
    for (int i = 0; i < flowerbedSize; i++) {  
  
        if (flowerbed[i] == 0 && (i == 0 || flowerbed[i - 1] == 0) &&  
  
            (i == flowerbedSize - 1 || flowerbed[i + 1] == 0)) {  
  
            flowerbed[i] = 1;  
  
            n--;  
  
        }  
  
    }  
  
    if (n <= 0)  
  
        return true;  
  
    else  
  
        return false;  
  
}
```

The screenshot shows a code editor interface for a C++ problem. The code is as follows:

```
1 bool canPlaceFlowers(int* flowerbed, int flowerbedSize, int n) {  
2     for (int i = 0; i < flowerbedSize; i++) {  
3         if (flowerbed[i] == 0 && (i == 0 || flowerbed[i - 1] == 0) &&  
4             (i == flowerbedSize - 1 || flowerbed[i + 1] == 0)) {  
5             flowerbed[i] = 1;  
6         }  
7     }  
8     if (n <= 0) return true;  
9     else return false;  
10 }
```

The editor shows the code is "Accepted" with 130/130 testcases passed. The runtime is 0 ms and memory usage is 10.08 MB. The test case input is:

flowerbed = [1, 0, 0, 0, 1]
n = 1

Week – 7

1.Knapsack(0/1)

Implement the 0/1 Knapsack problem using dynamic programming.

C Code:-

```
#include <stdio.h>

int max(int a, int b)

{

    return (a > b) ? a : b;

}

int main()

{

    int n, W;

    scanf("%d", &n);

    int wt[n], val[n];

    for(int i = 0; i < n; i++)

        scanf("%d", &wt[i]);

    for(int i = 0; i < n; i++)

        scanf("%d", &val[i]);

    scanf("%d", &W);

    int dp[n + 1][W + 1];

    for(int i = 0; i <= n; i++)

    {

        for(int j = 0; j <= W; j++)

        {

            if(i == 0 || j == 0)

                dp[i][j] = 0;
```

```

else if(wt[i - 1] <= j)

    dp[i][j] = max(val[i - 1] + dp[i - 1][j - wt[i - 1]],

        dp[i - 1][j]);

else

    dp[i][j] = dp[i - 1][j];

}

}

printf("%d\n", dp[n][W]);

return 0;

}

```

Output:-

4

1 3 4 5

1 4 5 7

7

9

2.Target Sum(Leetcode – 494)

You are given an integer array `nums` and an integer `target`.

You want to build an **expression** out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers.

For example, if `nums` = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different **expressions** that you can build, which evaluates to `target`.

Example 1:

Input: `nums` = [1,1,1,1,1], `target` = 3

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of `nums` be `target` 3.

$$-1 + 1 + 1 + 1 + 1 = 3$$

$$+1 - 1 + 1 + 1 + 1 = 3$$

$$+1 + 1 - 1 + 1 + 1 = 3$$

$$+1 + 1 + 1 - 1 + 1 = 3$$

$$+1 + 1 + 1 + 1 - 1 = 3$$

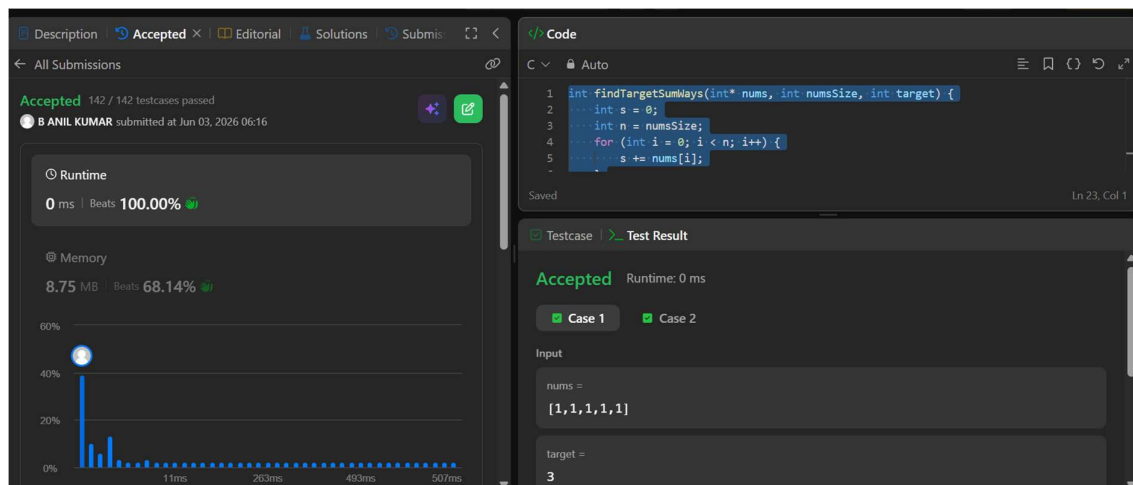
Example 2:

Input: `nums` = [1], `target` = 1

Output: 1

C Code:-

```
int findTargetSumWays(int* nums, int numsSize, int target) {  
    int s = 0;  
    int n = numsSize;  
    for (int i = 0; i < n; i++) {  
        s += nums[i];  
    }  
    if ((s + target) % 2 != 0 || abs(target) > s) {  
        return 0;  
    }  
    int k = (s + target) / 2;  
    int dp[k + 1];  
    for (int i = 0; i <= k; i++) {  
        dp[i] = 0;  
    }  
    dp[0] = 1;  
    for (int i = 0; i < n; i++) {  
        for (int j = k; j >= nums[i]; j--) {  
            dp[j] += dp[j - nums[i]];  
        }  
    }  
    return dp[k];  
}
```



Week – 8

1.Heap Sort

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

C Code:-

```
#include <stdio.h>

#include <time.h>

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--)
```

```

    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    clock_t start, end;

    start = clock();

    heapSort(arr, n);

    end = clock();

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\n%f\n", time_taken);

    return 0;
}

```

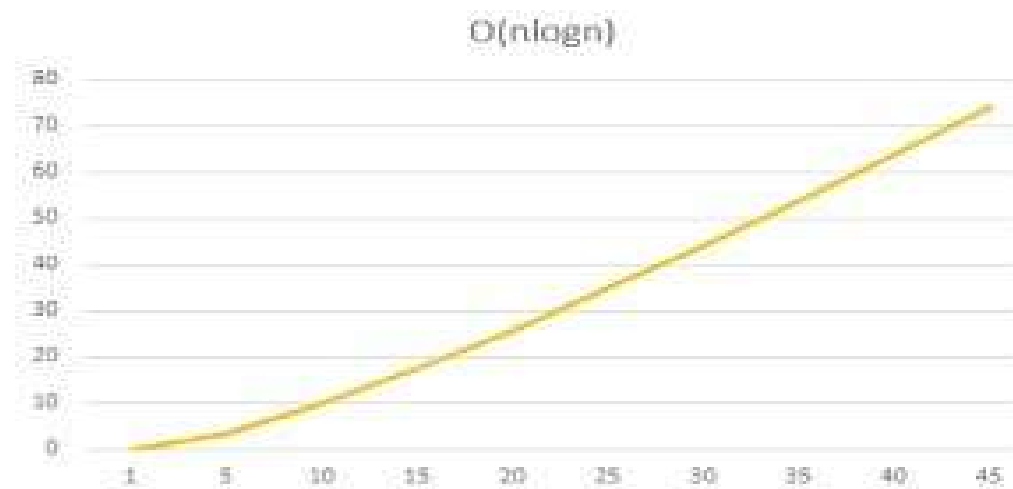
Output:-

6

12 11 13 5 6 7

5 6 7 11 12 13

0.000001



Week – 9

1.Topological Sort

Write program to obtain the Topological ordering of vertices in a given digraph.

C Code:-

```
#include <stdio.h>

int main()
{
    int n, e;
    scanf("%d %d", &n, &e);
    int adj[n][n];
    int indegree[n];
    for(int i = 0; i < n; i++)
    {
        indegree[i] = 0;
        for(int j = 0; j < n; j++)
            adj[i][j] = 0;
    }
    for(int i = 0; i < e; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        indegree[v]++;
    }
    int queue[n];
    int front = 0, rear = 0;
    for(int i = 0; i < n; i++)
    {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }
    while(front < rear)
```

```

{
    int u = queue[front++];
    printf("%d ", u);
    for(int v = 0; v < n; v++)
    {
        if(adj[u][v] == 1)
        {
            indegree[v]--;
            if(indegree[v] == 0)
                queue[rear++] = v;
        }
    }
}
return 0;
}

```

Output:-

6 6

5 2

5 0

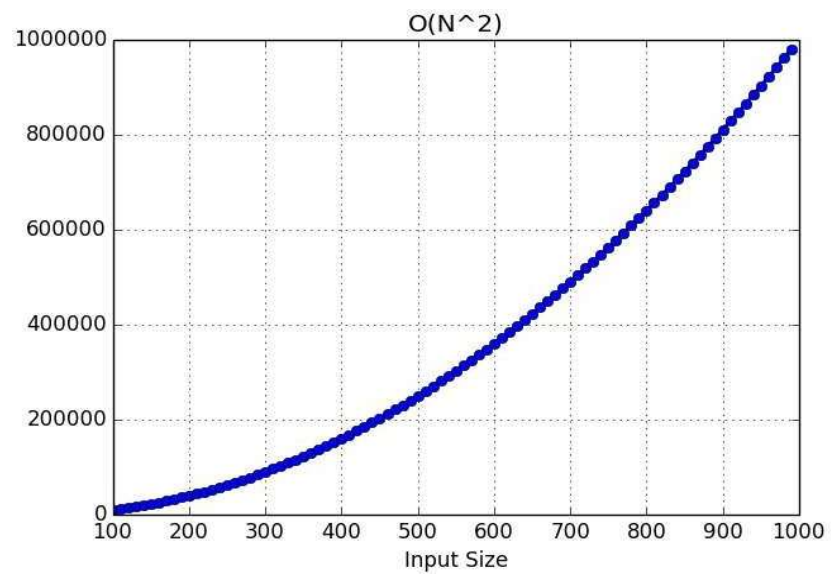
4 0

4 1

2 3

3 1

4 5 2 0 3 1



2.Course Schedule (Leetcode – 207)

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you **must** take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

C Code:-

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(numCourses * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][graphSize[b]++] = a;
        indegree[a]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    int count = 0;
    while (front < rear) {
        int node = queue[front++];
        count++;
        for (int i = 0; i < graphSize[node]; i++) {
            int neighbor = graph[node][i];
            indegree[neighbor]--;
            if (indegree[neighbor] == 0) {
                queue[rear++] = neighbor;
            }
        }
    }
}
```



```

    }

    return count == numCourses;
}

```

The screenshot shows a LeetCode submission interface. On the left, the submission status is 'Accepted' with 54/54 testcases passed. The user 'ANIL KUMAR' submitted it on Jun 03, 2026 at 06:26. The runtime is 35 ms (Beats 41.18%) and memory usage is 109.94 MB (Beats 32.32%). A bar chart shows the distribution of runtime performance across other submissions.

The code editor on the right shows the following C++ code:

```

1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
  prerequisitesColSize) {
2     int* indegree = (int*)calloc(numCourses, sizeof(int));
3
4     int** graph = (int**)malloc(numCourses * sizeof(int*));

```

The test result section shows 'Accepted' with a runtime of 0 ms. Two test cases are listed: Case 1 and Case 2. The input for Case 1 is:

```

numCourses = 2
prerequisites = [[1,0]]

```

Week – 10

1.Kruskal's Algorithm

Find the minimum cost spanning tree of a given undirected graph using Kruskal's algorithm.

C Code:-

```
#include <stdio.h>

struct Edge

{

    int u, v, w;

};

int parent[100];

int find(int i)

{

    while (parent[i] != i)

        i = parent[i];

    return i;

}

void unionSet(int a, int b)

{

    parent[a] = b;

}

int main()

{

    int n, e;

    scanf("%d %d", &n, &e);

    struct Edge edge[e];

    for (int i = 0; i < e; i++)
```

```

scanf("%d %d %d", &edge[i].u, &edge[i].v, &edge[i].w);

for (int i = 0; i < n; i++)

    parent[i] = i;

for (int i = 0; i < e - 1; i++)

{
    for (int j = 0; j < e - i - 1; j++)

    {
        if (edge[j].w > edge[j + 1].w)

        {
            struct Edge temp = edge[j];

            edge[j] = edge[j + 1];

            edge[j + 1] = temp;

        }

    }

}

int minCost = 0;

for (int i = 0; i < e; i++)

{

    int a = find(edge[i].u);

    int b = find(edge[i].v);

    if (a != b)

    {

        printf("%d %d %d\n",

            edge[i].u,

            edge[i].v,

            edge[i].w);

```

```
        minCost += edge[i].w;

        unionSet(a, b);

    }

}

printf("%d\n", minCost);

return 0;

}
```

Output:-

4 5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

2 3 4

0 3 5

0 1 10

19

2.Prim's Algorithm

Find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

C Code:-

```
#include <stdio.h>

#include <limits.h>

int main()
{
    int n;

    scanf("%d", &n);

    int cost[n][n];

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(i != j && cost[i][j] == 0)
                cost[i][j] = INT_MAX;
        }
    }

    int visited[n];

    for(int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;

    int edges = 0;

    int mincost = 0;
```

```

while(edges < n - 1)
{
    int min = INT_MAX;

    int a = -1, b = -1;

    for(int i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(int j = 0; j < n; j++)
            {
                if(!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];

                    a = i;

                    b = j;
                }
            }
        }
    }

    printf("%d %d %d\n", a, b, min);

    mincost += min;

    visited[b] = 1;

    edges++;
}

printf("%d\n", mincost);

```

```
    return 0;  
}
```

Output:-

4

0 10 6 5

10 0 0 15

6 0 0 4

5 15 4 0

0 3 5

3 2 4

0 1 10

19

Week-11

i)Implement Johnson Trotter algorithm to generate permutations.

C Code:-

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0, mobilePos = -1;
    for(int i = 0; i < n; i++)
    {
        if(dir[i] == LEFT && i != 0 && a[i] > a[i - 1])
        {
            if(a[i] > mobile)
            {
                mobile = a[i];
                mobilePos = i;
            }
        }
        if(dir[i] == RIGHT && i != n - 1 && a[i] > a[i + 1])
        {
            if(a[i] > mobile)
            {
                mobile = a[i];
                mobilePos = i;
            }
        }
    }
    return mobilePos;
}

void printPermutation(int a[], int n)
{

```



```

        for(int i = 0; i < n; i++)
            printf("%d ", a[i]);
        printf("\n");
    }
int main()
{
    int n;
    scanf("%d", &n);
    int a[n], dir[n];
    for(int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(a, n);
    while(1)
    {
        int mobilePos = getMobile(a, dir, n);
        if(mobilePos == -1)
            break;
        int swapPos;
        if(dir[mobilePos] == LEFT)
            swapPos = mobilePos - 1;
        else
            swapPos = mobilePos + 1;
        int temp = a[mobilePos];
        a[mobilePos] = a[swapPos];
        a[swapPos] = temp;
        temp = dir[mobilePos];
        dir[mobilePos] = dir[swapPos];
        dir[swapPos] = temp;
        mobilePos = swapPos;
    }
}

```

```
int mobile = a[mobilePos];  
for(int i = 0; i < n; i++)  
{  
    if(a[i] > mobile)  
        dir[i] = -dir[i];  
}  
printPermutation(a, n);  
}  
return 0;  
}
```

Output:-

3

1 2 3

1 3 2

3 1 2

3 2 1

2 3 1

2 1 3

ii) Implement “N-Queens Problem” using Backtracking.

C Code:-

```
#include <stdio.h>
#include <stdlib.h>
int x[20];
int place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}
void nQueens(int k, int n)
{
    for(int i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;

            if(k == n)
            {
                for(int j = 1; j <= n; j++)
                    printf("%d ", x[j]);
                printf("\n");
            }
            else
            {
                nQueens(k + 1, n);
            }
        }
    }
}
```

```

        }
    }
}
}
int main()
{
    int n;
    scanf("%d", &n);
    nQueens(1, n);
    return 0;
}

```

Output:-

4

2 4 1 3

3 1 4 2